PUC Minas



Algoritmos e Estruturas de Dados I

Prof. Dr. João Paulo Aramuni

Aula 04

Abstrações de dados homogêneos

AEDS I - Manhã

04

Abstrações de dados homogêneos

4.1 Arranjos: tabelas, vetores e matrizes

4.2 Definições e acessos

Unidade de Ensino



4.1 Arranjos: tabelas, vetores e matrizes

Arrays (ou Arranjos) são estruturas de dados, na forma de um grupo de posições contíguas na memória, com valores de mesmo nome e mesmo tipo.

Os arrays são estruturas estáticas, isto é, uma vez criados mantêm seu tamanho inicial. Os elementos de um array são encontrados pela referência do nome do array e pelo número da posição em que se encontra (também chamada índice).



4.1 Arranjos: tabelas, vetores e matrizes

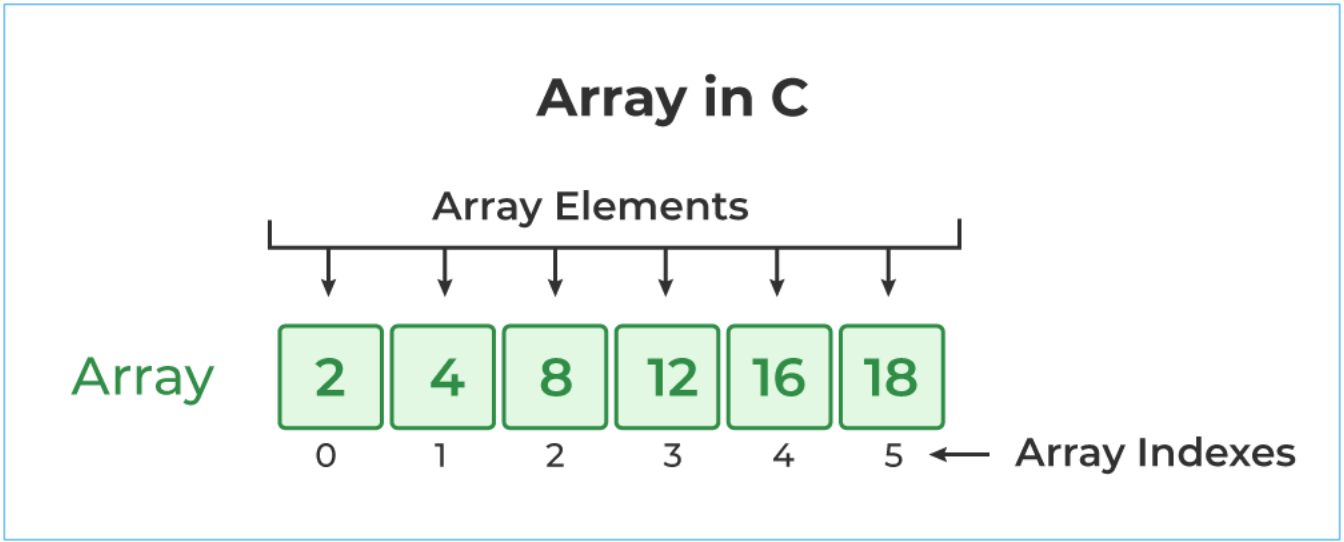
Arrays em C iniciam pela posição zero.

Portanto, um array c de três elementos tem as posições c[0], c[1] e c[2]. Para se encontrar o elemento de um array se usa o nome do array, seguido do índice (ou seja, a posição desejada), entre colchetes.

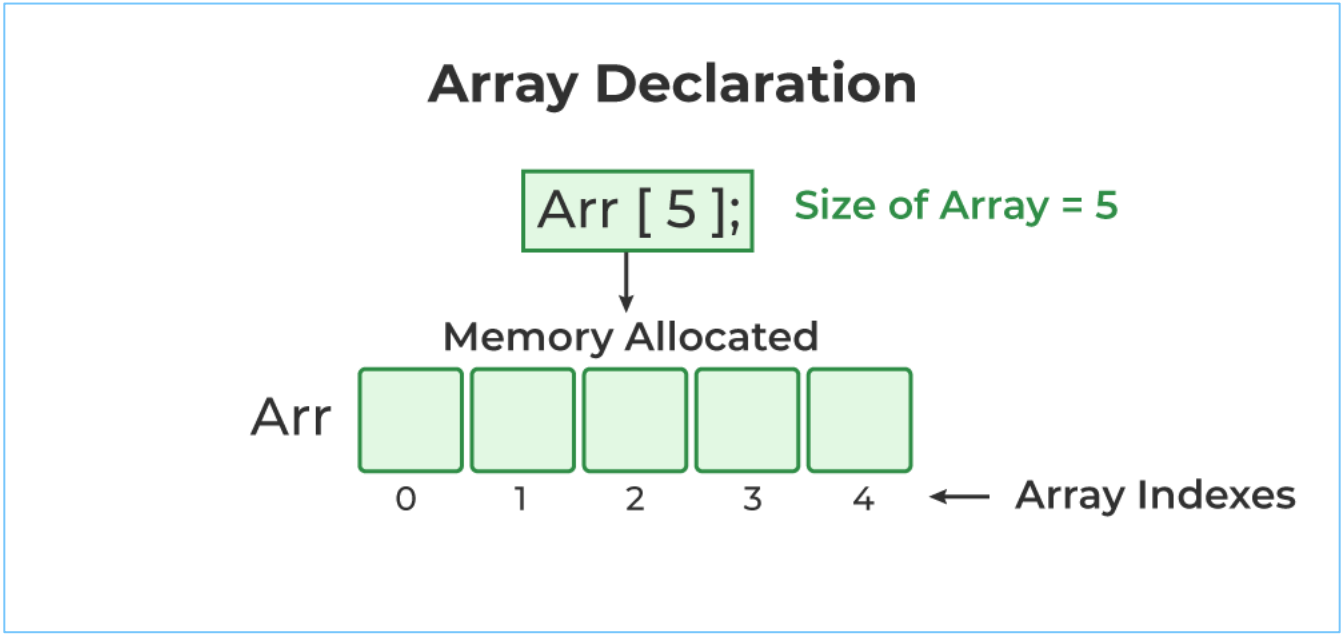
Exemplo: int numeros[5] = { 1, 2, 3, 4, 5 };



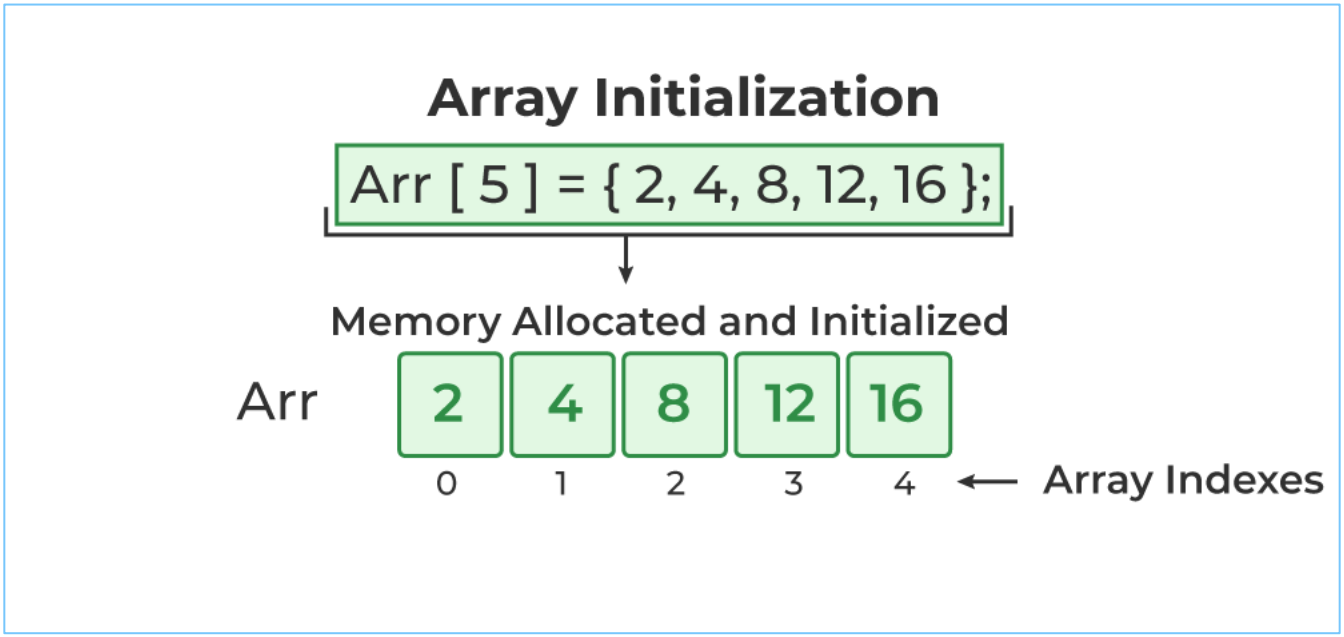
4.1 Arranjos: tabelas, vetores e matrizes



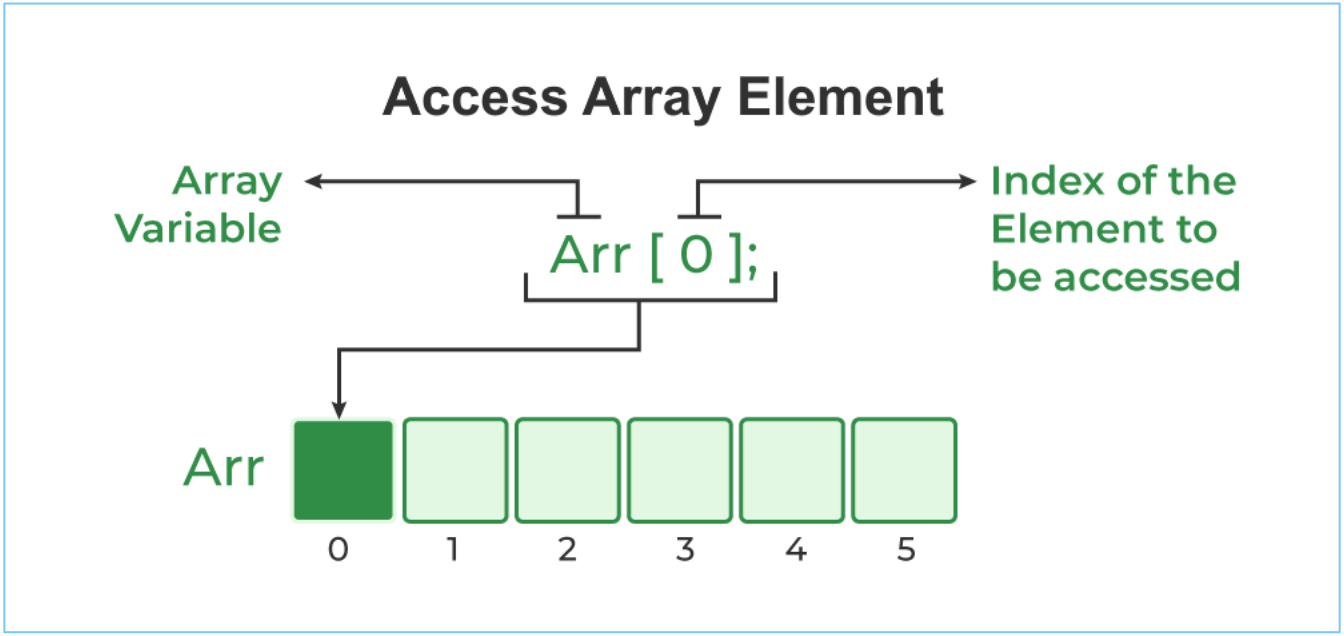
4.1 Arranjos: tabelas, vetores e matrizes



4.1 Arranjos: tabelas, vetores e matrizes



4.1 Arranjos: tabelas, vetores e matrizes



4.1 Arranjos: tabelas, vetores e matrizes

Exemplo 1:

```c
int main(void) {
    // Exemplo de declaração e inicialização de um array de inteiros
    int numeros[5] = { 1, 2, 3, 4, 5 };
    // Exemplo de declaração e inicialização de um array de strings
    char clubes[][30] = { "Atlético Mineiro", "Atlético Paranaense",
            "Atlético de Madrid" };

    // Loop for para percorrer o array e imprimir seus elementos
    printf("Elementos do array de inteiros: ");
    for (int i = 0; i < 5; i++) {
        printf("%d ", numeros[i]);
    }

    // Loop for para percorrer o array de strings e imprimir os nomes dos clubes
    printf("\nNomes dos clubes: ");
    for (int i = 0; i < 3; i++) {
        printf("%s ", clubes[i]);
    }

    return 0;
}
```

Console | Problems | Properties | Search | Tasks

```
<terminated> (exit value: 0) Prj_Array [C/C++ Application] /Users/joaopauloaramuni/Documents/ECLIPSE-WORKSPACE/Prj_Array_Ex1/Debug/Prj_Array_Ex1 (14/02/2024 14:16)
Elementos do array de inteiros: 1 2 3 4 5
Nomes dos clubes: Atlético Mineiro Atlético Paranaense Atlético de Madrid
```

4.1 Arranjos: tabelas, vetores e matrizes

Exemplo 2:

```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

int main(void) {

    // Declaração de um array de strings para armazenar três nomes
    char nomes[3][30];  // Supondo nomes com até 30 caracteres

    // Solicitar ao usuário para inserir três nomes
    for (int i = 0; i < 3; i++) {
        printf("Digite o %do nome: ", i + 1);
        scanf("%s", nomes[i]);
    }

    // Imprimir os nomes na ordem inversa
    printf("\nNomes na ordem inversa:\n");
    for (int i = 2; i >= 0; i--) {
        printf("%s\n", nomes[i]);
    }
```

4.1 Arranjos: tabelas, vetores e matrizes

Arrays nada mais são do que **listas**.

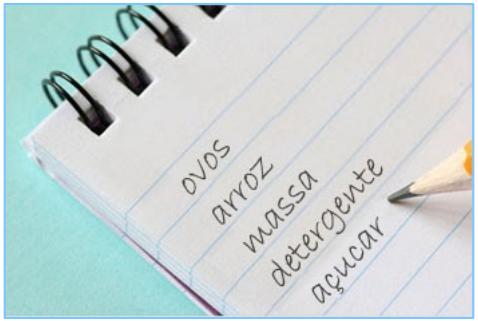
O conceito de lista é algo muito comum para as pessoas. Trata-se de uma relação finita de itens, todos eles contidos dentro de um mesmo tema.

Vários são os exemplos possíveis de listas: itens em estoque em uma empresa, dias da semana, lista de compras do supermercado, convidados de uma festa etc.

4.1 Arranjos: tabelas, vetores e matrizes

Em ciência da computação, uma lista é uma estrutura de dados linear utilizada para armazenar e organizar dados em um computador.

Uma estrutura do tipo lista é uma sequência de elementos do mesmo tipo. Seus elementos possuem estrutura interna abstraída, ou seja, sua complexidade é arbitrária e não afeta o seu funcionamento.



4.1 Arranjos: tabelas, vetores e matrizes

Além disso, uma lista pode possuir elementos repetidos, assim como ser ordenada ou não, dependendo da aplicação.

Como nas listas que conhecemos, a estrutura do tipo lista pode possuir N (N>=0) elementos ou itens. Se N = 0, dizemos que a lista está **vazia**.

Leitura sugerida:

https://www.geeksforgeeks.org/c-arrays/

4.1 Arranjos: tabelas, vetores e matrizes

Existem várias representações diferentes para uma lista.

Essas representações variam a depender de como os elementos são inseridos ou removidos da lista, tipo de alocação usada e tipo de acesso aos elementos.

Vejamos alguns tipos clássicos de lista.

4.1 Arranjos: tabelas, vetores e matrizes

Quanto à inserção/remoção de elementos da lista, temos:

- **lista convencional**: pode ter elementos inseridos ou removidos de qualquer lugar dela;
- **fila**: estrutura do tipo FIFO (*First In First Out*), os elementos só podem ser inseridos no final, e acessados ou removidos do início da lista.
- **pilha**: estrutura do tipo LIFO (*Last in First Out*), os elementos só podem ser inseridos, acessados ou removidos do final da lista.

...

4.1 Arranjos: tabelas, vetores e matrizes

Quanto à alocação de memória, podemos utilizar alocação estática ou dinâmica para implementar uma lista:

- **alocação estática**: o espaço de memória é alocado no momento da compilação do programa. É necessário definir do número máximo de elementos que a lista irá possuir;
- **alocação dinâmica**: o espaço de memória é alocado em tempo de execução. A lista cresce à medida que novos elementos são armazenados, e diminui à medida que elementos são removidos.

4.1 Arranjos: tabelas, vetores e matrizes

E, independentemente de como a memória foi alocada, podemos acessar os seus elementos de duas formas:

- **acesso sequencial**: os elementos são armazenados de forma consecutiva na memória (como em um array ou vetor). A posição de um elemento pode ser facilmente obtida a partir do início da lista;
- **acesso encadeado**: cada elemento pode estar em uma área distinta da memória, não necessariamente consecutivas. É necessário que cada elemento da lista armazene, além da sua informação, o endereço de memória onde se encontra o próximo elemento. Para acessar um elemento, é preciso percorrer todos os seus antecessores na lista.

4.1 Arranjos: tabelas, vetores e matrizes

Independentemente do tipo de alocação e acesso usado na implementação de uma lista, as seguintes operações básicas são possíveis:

- criação da lista;
- inserção de um elemento na lista;
- remoção de um elemento da lista;
- busca por um elemento da lista;
- destruição da lista;
- além de informações como tamanho, se está cheia ou vazia.

4.1 Arranjos: tabelas, vetores e matrizes

A operação de **inserção** é o ato de guardar elementos dentro da lista.

Existem 3 tipos de inserção:

- inserção no início;
- no final ou;
- no meio (isto é, entre dois elementos) da lista.

4.1 Arranjos: tabelas, vetores e matrizes

A operação de inserção no meio da lista é comumente usada quando se deseja inserir um elemento de forma ordenada na lista.

A operação de inserção envolve o teste de estouro da lista, ou seja, precisamos verificar se é possível inserir um novo elemento na lista (a lista ainda não está cheia).

4.1 Arranjos: tabelas, vetores e matrizes

Existindo uma lista e ela possuindo elementos, é possível excluí-los. E é disso que se trata a operação de remoção.

Existem 3 tipos de remoção:

- remoção do início;
- do final ou;
- do meio (isto é, entre entre dois elementos) da lista.

4.1 Arranjos: tabelas, vetores e matrizes

A operação de remoção do meio da lista é comumente usada quando se deseja remover um elemento específico da lista.

A operação de remoção envolve o teste de lista vazia, ou seja, precisamos verificar se existem elementos dentro da lista antes de tentar removê-los.

4.1 Arranjos: tabelas, vetores e matrizes

O termo "**vetor**" é frequentemente usado como sinônimo de "**array**", especialmente quando se refere a arrays unidimensionais.

Então, quando você ouvir ou ler sobre "vetores" em C, geralmente estão se referindo a estruturas de dados que são arrays.

```
int numeros[5] = { 1, 2, 3, 4, 5 };
```

4.1 Arranjos: tabelas, vetores e matrizes

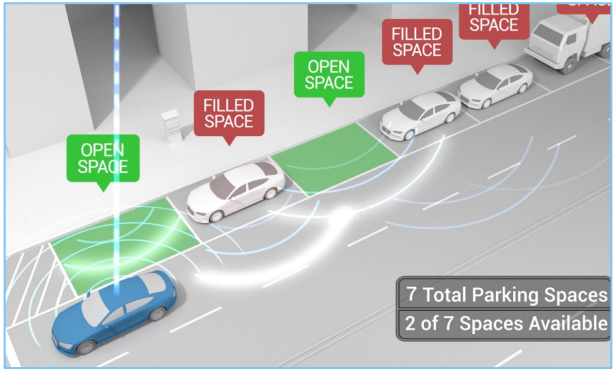
No entanto, vale a pena notar que o termo "vetor" pode ser mais frequentemente usado em ambientes educacionais ou informais, enquanto a documentação técnica ou códigos mais formais podem preferir o termo "array".

Ambos se referem à mesma ideia de uma coleção de elementos do mesmo tipo armazenados em posições contíguas de memória.

4.1 Arranjos: tabelas, vetores e matrizes

Vamos considerar um exemplo prático de quando usar um array unidimensional.

Exemplo: Um array unidimensional para representar a quantidade de vagas disponíveis em um estacionamento ao longo do tempo



4.1 Arranjos: tabelas, vetores e matrizes

Imagine um programa que simula a administração de um estacionamento de carros utilizando um array unidimensional.

O estacionamento tem um número fixo de vagas, representadas por um array unidimensional. Cada posição no array representa uma vaga do estacionamento.

4.1 Arranjos: tabelas, vetores e matrizes

Array unidimensional (estacionamento):

Um array que armazena informações sobre cada vaga do estacionamento. Cada elemento no array representa uma posição onde um carro pode estar ou uma vaga livre.

Iniciação do estacionamento:

Ao iniciar, o programa inicializa o array estacionamento, atribuindo um valor padrão (por exemplo, espaço vazio ' ') a cada posição, indicando que todas as vagas estão inicialmente livres.

4.1 Arranjos: tabelas, vetores e matrizes

Representação de carros no estacionamento:

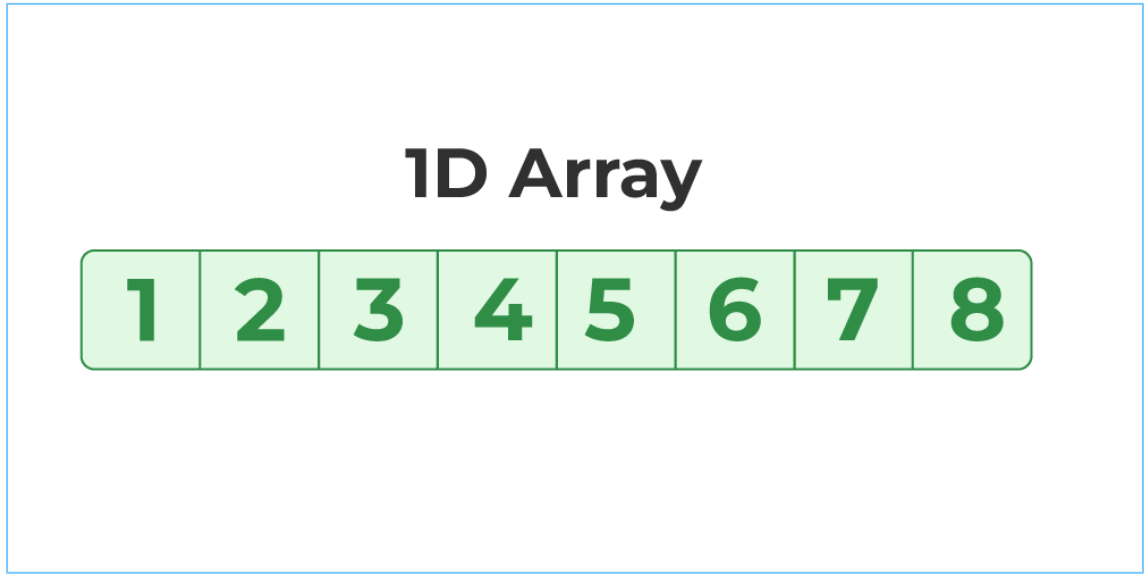
O programa pode simular a presença de carros atribuindo o caractere 'C' a posições específicas do array estacionamento. Cada 'C' representa a presença de um carro em uma vaga específica.

Exibição do estado do estacionamento:

O estado atual do estacionamento pode ser visualizado percorrendo o array estacionamento e exibindo os caracteres presentes em cada posição. Um espaço vazio (' ') indica uma vaga livre, enquanto um 'C' indica a presença de um carro na vaga correspondente.

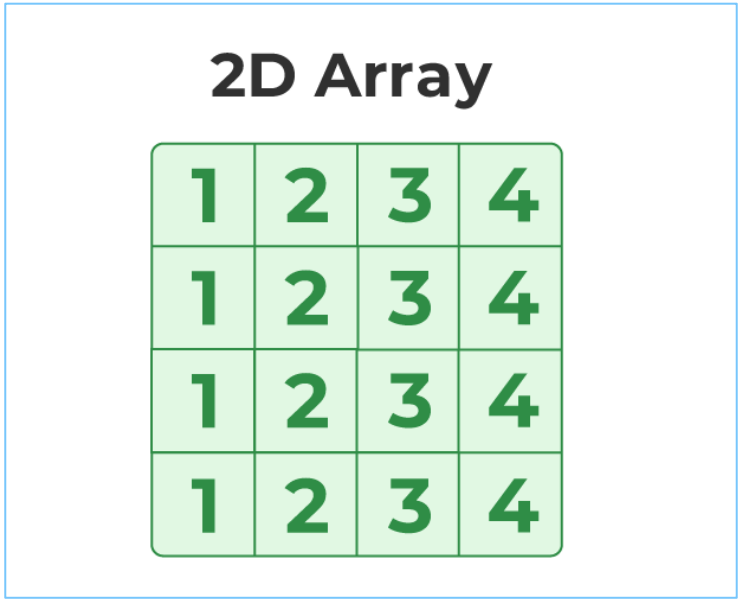
4.1 Arranjos: tabelas, vetores e matrizes

Array unidimensional (ou **vetor**):



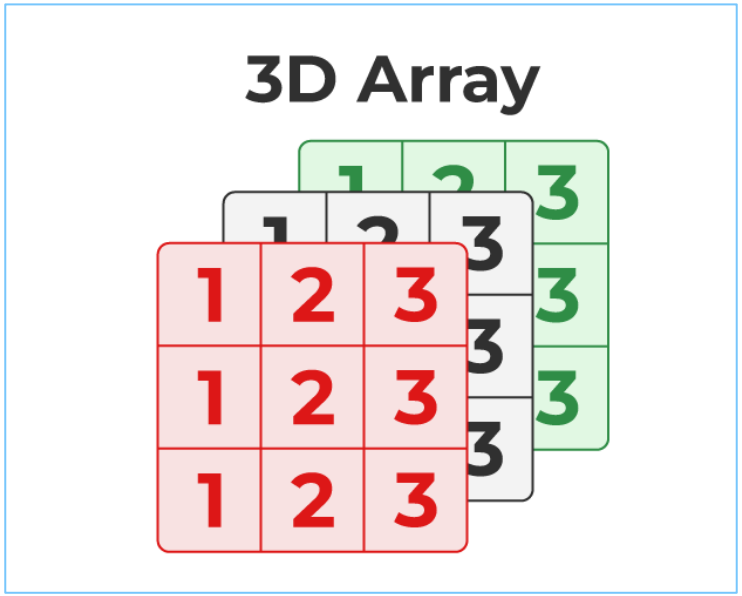
4.1 Arranjos: tabelas, vetores e matrizes

Array bidimensional (ou **matriz**):



4.1 Arranjos: tabelas, vetores e matrizes

Array tridimensional:



4.1 Arranjos: tabelas, vetores e matrizes

Em termos gerais, "array bidimensional" e "**matriz**" são frequentemente usados como sinônimos em programação.

Ambos se referem a uma estrutura de dados que tem duas dimensões: linhas e colunas. Essa estrutura é organizada como uma tabela com entradas dispostas em linhas e colunas.

Exemplo: int matriz2D[3][4];

4.1 Arranjos: tabelas, vetores e matrizes

Neste caso, matriz2D[3][4] é uma matriz bidimensional de inteiros com 3 linhas e 4 colunas. Portanto, em muitos contextos, os termos "array bidimensional" e "matriz" são intercambiáveis e referem-se à mesma estrutura de dados.

No entanto, é importante notar que, em alguns contextos, a palavra "matriz" pode ser usada de maneira mais específica para se referir a uma estrutura matemática, enquanto "array bidimensional" é um termo mais comum na programação.

4.1 Arranjos: tabelas, vetores e matrizes

Vamos considerar um exemplo prático de quando usar uma matriz bidimensional.

Exemplo: Uma matriz bidimensional para representar o tabuleiro de um jogo da velha.



4.1 Arranjos: tabelas, vetores e matrizes

Imagine um programa que simula um jogo da velha usando uma matriz bidimensional.

Cada célula da matriz representa uma posição no tabuleiro do jogo da velha.

As duas dimensões da matriz representam as linhas e colunas do tabuleiro.

4.1 Arranjos: tabelas, vetores e matrizes

Tamanho do tabuleiro de Jogo da Velha: 3x3 (tabuleiro bidimensional com 3 linhas e 3 colunas).

A matriz bidimensional tabuleiro seria usada para armazenar os marcadores dos jogadores nas posições do tabuleiro.

Vejamos como ficaria.

4.1 Arranjos: tabelas, vetores e matrizes

```
char tabuleiro[3][3] = {
 {'X', 'O', 'X'},
{'O', 'X', 'O'},
{'X', 'O', 'X'} };
```

Neste exemplo: 'X' representa o marcador do jogador X. 'O' representa o marcador do jogador O. Cada célula no tabuleiro (tabuleiro[i][j]) representa uma posição no jogo da velha, e os jogadores alternam marcando essas posições durante o jogo.

4.1 Arranjos: tabelas, vetores e matrizes

O estado inicial do tabuleiro pode ser qualquer configuração válida.

Cada elemento no array bidimensional representa uma célula do tabuleiro, e o conteúdo dessas células (X, O ou espaço vazio) muda à medida que os jogadores fazem seus movimentos.

O objetivo do jogo é conseguir uma linha, coluna ou diagonal com três marcadores iguais antes do adversário.

4.1 Arranjos: tabelas, vetores e matrizes

Veja abaixo um exemplo de implementação do jogo da velha, escrito em Linguagem C, utilizando um array bidimensional (matriz).

Construído no segundo semestre do ano de 2010, durante a disciplina de Estrutura de Dados I:

https://github.com/joaopauloaramuni/c/blob/main/ed1_velha/src/ed1_velha_JoaoPaulo.c

4.1 Arranjos: tabelas, vetores e matrizes

Exemplo de matriz bidimensional:

```c
#define LINHAS 3
#define COLUNAS 3

int main(void) {
    // Declaração da matriz bidimensional
    int matriz[LINHAS][COLUNAS];

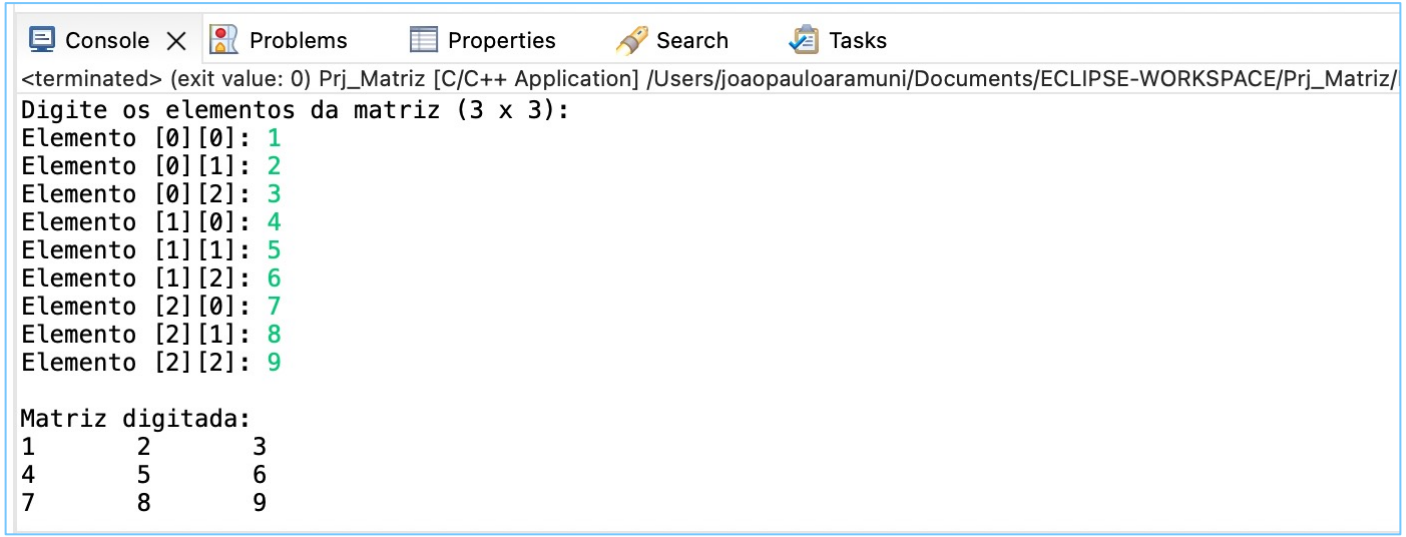
    // Preenchimento da matriz
    printf("Digite os elementos da matriz (%d x %d):\n", LINHAS, COLUNAS);
    for (int i = 0; i < LINHAS; i++) {
        for (int j = 0; j < COLUNAS; j++) {
            printf("Elemento [%d][%d]: ", i, j);
            scanf("%d", &matriz[i][j]);
        }
    }

    // Exibição da matriz
    printf("\nMatriz digitada:\n");
    for (int i = 0; i < LINHAS; i++) {
        for (int j = 0; j < COLUNAS; j++) {
            printf("%d\t", matriz[i][j]);
        }
        printf("\n");
    }

    return 0;
}
```

4.1 Arranjos: tabelas, vetores e matrizes

Exemplo de matriz bidimensional:



4.1 Arranjos: tabelas, vetores e matrizes

Uma matriz tridimensional em C é uma estrutura de dados que possui três dimensões, ou seja, é organizada em três índices.

Altura, largura e profundidade representam o número de elementos em cada dimensão da matriz.

Exemplo: int matriz3D[2][3][4];

4.1 Arranjos: tabelas, vetores e matrizes

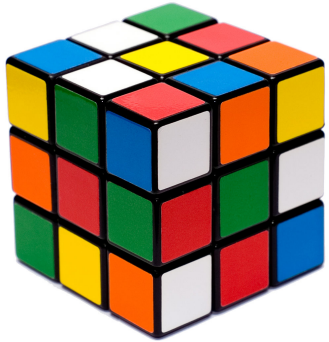
```
int matriz3D[2][3][4] = {
{ {1, 2, 3, 4},
{5, 6, 7, 8},
{9, 10, 11, 12} },
{ {13, 14, 15, 16},
{17, 18, 19, 20},
{21, 22, 23, 24} } };
```

A altura é 2 (duas "camadas"). A largura é 3 (três "linhas"). A profundidade é 4 (quatro "colunas"). Portanto, a matriz tridimensional matriz3D tem 2 camadas, cada camada tem 3 linhas, e cada linha tem 4 colunas.

4.1 Arranjos: tabelas, vetores e matrizes

Vamos considerar um exemplo prático de quando usar uma matriz tridimensional.

Exemplo: Uma matriz tridimensional para representar o "Cubo de Rubik", um quebra-cabeça tridimensional icônico.



4.1 Arranjos: tabelas, vetores e matrizes

Imagine um programa que simula um "Cubo de Rubik" usando uma matriz tridimensional.

Cada célula do cubo (posição no array tridimensional) representa um pequeno bloco colorido do Cubo de Rubik.

As três dimensões da matriz representariam as faces do cubo.

4.1 Arranjos: tabelas, vetores e matrizes

Tamanho do cubo de Rubik: 3x3x3 (cubo tridimensional com 3 unidades em cada dimensão).

Cada célula armazena: A cor do bloco.

Você poderia representar um estado inicial do Cubo de Rubik atribuindo cores a cada bloco nas diferentes posições.

4.1 Arranjos: tabelas, vetores e matrizes

```
char cubo_rubik[3][3][3] = {
{ {'R', 'R', 'W'}, {'G', 'G', 'W'}, {'B', 'O', 'O'} },
{ {'G', 'R', 'B'}, {'Y', 'W', 'Y'}, {'B', 'O', 'O'} },
 { {'G', 'R', 'Y'}, {'Y', 'Y', 'Y'}, {'O', 'W', 'O'} } };
```

Neste exemplo:

'R' representa a cor vermelha,
'G' representa a cor verde,
'B' representa a cor azul,
'W' representa a cor branca,
'Y' representa a cor amarela, e
'O' representa a cor laranja.

4.1 Arranjos: tabelas, vetores e matrizes

Cada posição no cubo (cubo_rubik[i][j][k]) representa a cor do bloco em uma determinada coordenada x, y, z do Cubo de Rubik.

Este é um exemplo simplificado e estático, mas em uma aplicação mais complexa, você poderia representar as rotações e movimentos do Cubo de Rubik por meio de manipulações na matriz tridimensional.

4.2 Definições e acessos

Definições formais:

Array unidimensional/Vetor/1D:

tipo_do_array nome_do_array[tamanho_do_array];

Array bidimensional/Matriz/2D:

tipo_do_array nome_do_array[linhas][colunas];

Array tridimensional/Matriz/3D:

tipo_do_array nome_do_array[altura][largura][profundidade];



4.2 Definições e acessos

Declarações:

Array unidimensional/Vetor/1D:

int array[5] = {1, 2, 3, 4, 5}

Array bidimensional/Matriz/2D:

int matriz2D[3][4] = { {1, 2, 3, 4}, {5, 6, 7, 8}, {9, 10, 11, 12} };

Array tridimensional/Matriz/3D:

int matriz3D[2][3][4] = { { {1, 2, 3, 4}, {5, 6, 7, 8}, {9, 10, 11, 12} },

{ {13, 14, 15, 16}, {17, 18, 19, 20}, {21, 22, 23, 24} } };



4.2 Definições e acessos

Como vimos, o **acesso** a elementos em arrays unidimensionais, bidimensionais e tridimensionais ocorre por meio do uso de índices.

Os índices indicam a posição de um elemento específico no array.

Vale lembrar que em muitas linguagens de programação, os índices começam em 0.

4.2 Definições e acessos

Vejamos três exemplos de acesso sequencial:

1. Acesso em array unidimensional: No caso do array unidimensional (vetor), o acesso é feito utilizando um único índice.

```
int array[5] = {1, 2, 3, 4, 5};
int elemento = array[2];
// Acessa o terceiro elemento (índice 2)
```

Neste exemplo, elemento terá o valor 3, pois o terceiro elemento do array é 3.

4.2 Definições e acessos

Vejamos três exemplos de acesso sequencial:

2. Acesso em array bidimensional (Matriz 2D): Para arrays bidimensionais, como matrizes, você utiliza dois índices para acessar um elemento.

```
int matriz2D[3][4] = { {1, 2, 3, 4}, {5, 6, 7, 8}, {9, 10, 11, 12} };
int elemento = matriz2D[1][2];
// Acessa o elemento na segunda linha e terceira coluna
```

Neste caso, elemento terá o valor 7, pois é o elemento na segunda linha e terceira coluna da matriz.

4.2 Definições e acessos

Vejamos três exemplos de acesso sequencial:

3. Acesso em array tridimensional (Matriz 3D): Para arrays tridimensionais, são necessários três índices para acessar um elemento.

```
int matriz3D[2][3][4] = { { {1, 2, 3, 4}, {5, 6, 7, 8}, {9, 10, 11, 12} },
{ {13, 14, 15, 16}, {17, 18, 19, 20}, {21, 22, 23, 24} } };
int elemento = matriz3D[1][2][0];
// Acessa o elemento na segunda matriz, terceira linha e primeira coluna
```

Neste exemplo, elemento terá o valor 21, pois é o elemento na segunda matriz, terceira linha e primeira coluna da matriz tridimensional.

Exercícios

Exercício 1) Array unidimensional

Enunciado: Crie um programa em C que solicite ao usuário que insira 5 números inteiros.

Em seguida, o programa deve calcular a soma e a média desses números e exibir os resultados na tela.

Exercícios

Exercício 2) Array bidimensional

Enunciado: Crie um programa em C que declare e inicialize uma matriz 3x3 de números inteiros.

Em seguida, o programa deve calcular e exibir a soma de todos os elementos da matriz.

Exercícios

Exercício 3) Array tridimensional

Enunciado: Crie um programa em C que declare e inicialize uma matriz tridimensional 2x2x2 de números inteiros.

Em seguida, o programa deve calcular e exibir a média dos elementos dessa matriz.

Disciplina – Algoritmos e Estruturas de Dados I

Referências básicas:

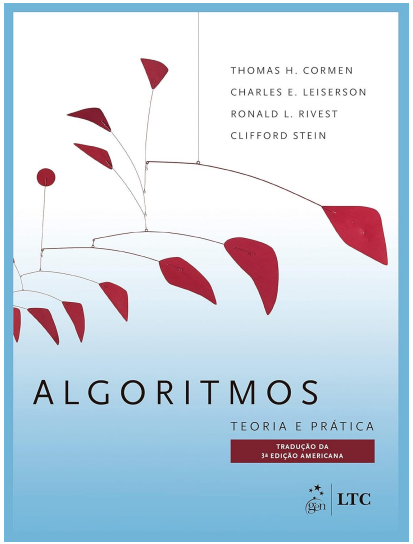
- CORMEN, Thomas. **Algoritmos** - Teoria e Prática. Editora GEN LTC. 3ª edição, 2012.

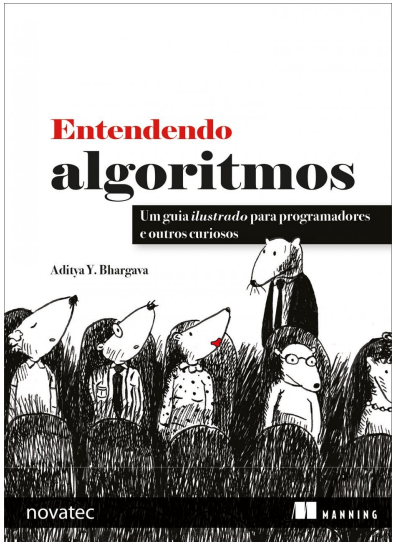
- BHARGAVA, Aditya Y. **Entendendo Algoritmos**: Um Guia Ilustrado Para Programadores e Outros Curiosos. Novatec Editora, 2017.

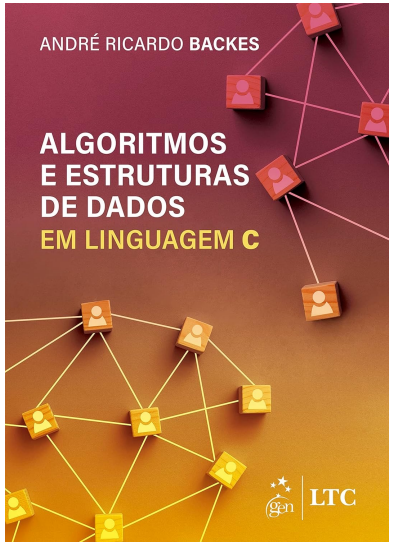
- BACKES, André Ricardo. **Algoritmos e Estruturas de Dados em Linguagem C**. Editora LTC, 2022.

Disciplina – Algoritmos e Estruturas de Dados I

Referências básicas:







Disciplina – Algoritmos e Estruturas de Dados I

Referências complementares:

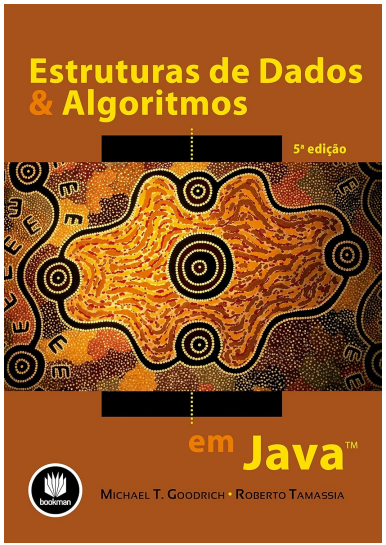
- GOODRICH, Michael T. **Estruturas de Dados e Algoritmos em Java**. Editora Bookman. 5ª edição, 2013.

- AGARWAL, Basant. **Estruturas de Dados e Algoritmos com Python**: Armazene, manipule e acesse dados de forma eficaz e melhore o desempenho de suas aplicações. Novatec Editora, 2023.

- GRONER, Loiane. **Estruturas de Dados e Algoritmos com JavaScript**: Escreva um Código JavaScript Complexo e Eficaz Usando a Mais Recente ECMAScript. Novatec Editora, 2019.

Disciplina – Algoritmos e Estruturas de Dados I

Referências complementares:







Disciplina – Algoritmos e Estruturas de Dados I

Outras referências:

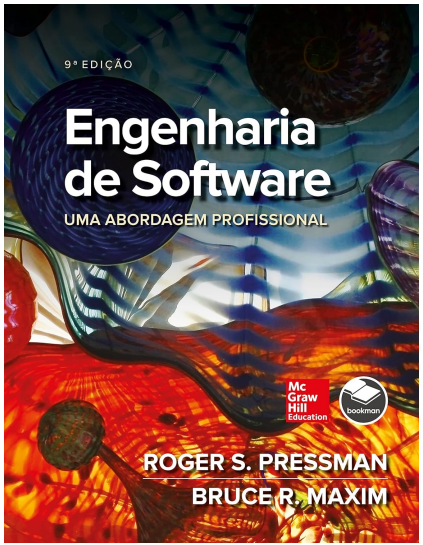
- PRESSMAN, Roger S.; MAXIM, Bruce R. **Engenharia de software**. Editora Grupo AMGH, 2021.

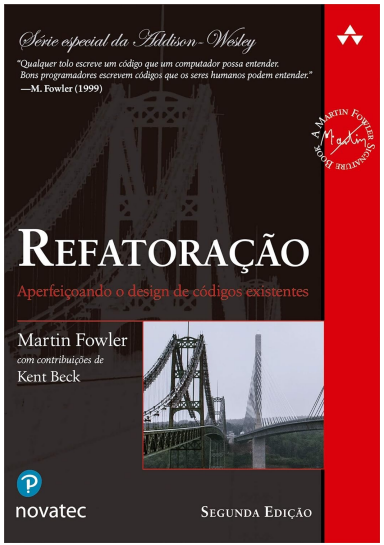
- FOWLER, Martin. **Refatoração**: Aperfeiçoando o Design de Códigos Existentes. Editora Novatec, 2020.

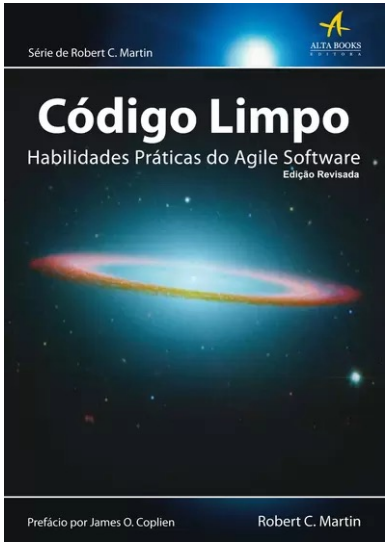
- MARTIN, Robert C. **Código limpo**: habilidades práticas do Agile software. Editora Alta Books, 2009.

Disciplina – Algoritmos e Estruturas de Dados I

Outras referências:







Disciplina – Algoritmos e Estruturas de Dados I

Nesta disciplina usaremos: Linguagem C (ANSI Style)

- Linguagem C
[DevDocs] https://devdocs.io/c/

- Eclipse IDE
[Download] https://www.eclipse.org/downloads/packages/release/2023-12/r/eclipse-ide-cc-developers
[Docs] https://www.eclipse.org/documentation/





Obrigado!

Dúvidas?

joaopauloaramuni@gmail.com





GitHub LinkedIn Lattes